

НА ЭТОЙ СТРАНИЦЕ

- Установка
- Официальная документация
- Базовая настройка
- Примеры
 - Генерация текста
 - Структурированный вывод
 - Ввод изображений / файлов
 - Потоковая передача
 - Вызов функций
 - Обоснование
 - Преко́нтекст
 - Цепочки
 - Задачи и ограничения
- Параметры клиента
- Асинхронность и пакетная обработка
- Ограничения сервера
- Ошибки
- Следующие шаги

Интеграция LangChain SDK

[копировать уценку](#)

Interfaze имеет встроенную интеграцию с LangChain. ChatInterfaze – это стандартная модель чата LangChain, которая, как и любая другая, встраивается в цепочки, агентов и LangGraph, а также отображает специфичные для Interfaze поля, которые не отображаются в обычной модели чата: precontext , reasoning , и vcache .

- [TypeScript / JavaScript SDK](#)
- [Python SDK](#)

Установка

машинописный... 

```
npm install @interfaze-ai/langchain
# or
yarn add @interfaze-ai/langchain
```



В TypeScript `@langchain/core`, `@langchain/openai`, и `interfaze` являются равноправными зависимостями. В примерах структурированного вывода и инструментов, приведенных ниже, также используется `zod`.

Официальная документация

- [Документация по LangChain JS](#)
- [Документация по LangChain Python](#)

Базовая настройка

Модель считывает `INTERFAZE_API_KEY` данные из окружения, поэтому ее можно создать без аргументов. Получите ключ API на [панели управления](#).

машинописный... ▾

LangChain SDK

```
import { ChatInterfaze } from "@interfaze-ai/langchain";

const interfaze = new ChatInterfaze({ apiKey: process.env.INTERFAZE_API_KEY });
```



- Не нужно настраивать базовый URL или название модели – и то, и другое встроено.
- Передаются обычные параметры LangChain (`temperature`, `maxTokens`, `timeout`, `reasoningEffort`). Тайм-аут запроса по умолчанию составляет 900 секунд, поскольку один вызов может включать в себя оптическое распознавание символов, веб-поиск или расшифровку.

Примеры

Генерация текста

машинописный... ▾

LangChain SDK

```
import { HumanMessage, SystemMessage } from "@langchain/core/messages";
```



```
const response = await interfaze.invoke([
  new SystemMessage("You are a helpful assistant."),
  new HumanMessage("Write a short story about a robot learning to paint"),
]);

console.log(response.content);
```

Структурированный вывод

Подробнее о [структурированном выводе](#).

машинописный... ▾

LangChain SDK

```
import { z } from "zod";

const weatherSchema = z.object({
  city: z.string().describe("The name of the city"),
  temperature_celsius: z.number().describe("Current temperature in Celsius"),
  condition: z.string().describe("Weather condition, e.g. sunny, rainy, cloudy"),
});

const structuredModel = interfaze.withStructuredOutput(weatherSchema);

const result = await structuredModel.invoke("What is the current weather in Tokyo");

console.log(result);
```

Чтобы сохранить исходные метаданные ответа, запросите исходное сообщение и считайте из него `precontext`.

машинописный... ▾

LangChain SDK

```
import { AIMessage, HumanMessage } from "@langchain/core/messages";
import { z } from "zod";

const IDSchema = z.object({
  first_name: z.string().describe("First name on the ID"),
  last_name: z.string().describe("Last name on the ID"),
  dob: z.string().describe("Date of birth on the ID"),
  driver_licence_number: z.string().describe("Driver licence number on the ID"),
});

const response = await interfaze.withStructuredOutput(IDSchema, { includeRaw: tr
```

```
new HumanMessage({
  content: [
    { type: "text", text: "Extract the details from this ID" }
  ]
})
```

Ввод изображения / файла

Изображения, аудио, PDF, документы Word и CSV – все они используют стандартные контентные части LangChain по URL или в формате base64. Подробнее об [обработке файлов](#).

машинописный... ▾

LangChain SDK

```
import { HumanMessage } from "@langchain/core/messages";

const response = await interfaze.invoke([
  new HumanMessage({
    content: [
      { type: "text", text: "What is in this image?" },
      {
        type: "image_url",
        image_url: {
          url: "https://r2public.jigsawstack.com/interfaze/examples/construction"
        },
      },
    ],
  }),
]);
```

В видео используется специфичный для Interfaze блок `video`, который принимает `url` или `base64`:

машинописный... ▾

LangChain SDK

```
import { HumanMessage } from "@langchain/core/messages";

const response = await interfaze.invoke([
  new HumanMessage({
    content: [
      { type: "text", text: "What happens in this clip?" },
      { type: "video", url: "https://r2public.jigsawstack.com/interfaze/examples" },
    ] as never,
  }),
]);

console.log(response.content);
```

Потоковая передача

Interface передает рассуждения и преконтекст в виде встроенных побочных каналов и ChatInterface удаляет их из потокового контента. Подробнее о [потоковой передаче](#).

машинописный... ▾

LangChain SDK

```
import { HumanMessage } from "@langchain/core/messages";

const stream = await interface.stream([new HumanMessage("Write a short story about a cat who likes to eat fish.")]);

for await (const chunk of stream) {
  process.stdout.write(chunk.content as string);
}
```

Function Calling

Bind tools with `bindTools`, then read `tool_calls` off the response. Learn more about [function calling](#).

typescript ▾

LangChain SDK

```
import { HumanMessage, ToolMessage } from "@langchain/core/messages";
import { tool } from "@langchain/core/tools";
import { z } from "zod";

// Step 1: Define tools
const getHoroscope = tool(
  async ({ sign }) => `Today's horoscope for ${sign}: You will have a great day!`,
  {
    name: "get_horoscope",
    description: "Get today's horoscope for an astrological sign.",
    schema: z.object({
      sign: z.string().describe("An astrological sign like Taurus or Aquarius"),
    }),
  },
);
```

Reasoning

Reasoning is off by default. Turn it on with `reasoningEffort`, then read the reasoning text off the response metadata. Learn more about [reasoning](#).

typescript ▾

LangChain SDK

```
const response = await interfaze.invoke("Which region should we launch in first",  
  reasoningEffort: "high",  
});  
  
console.log(response.response_metadata.reasoning);  
console.log(response.content);
```

Precontext

Every response carries the raw output of any task Interfaze ran behind the scenes, such as bounding boxes, OCR text, or search results. Learn more about [precontext](#).

typescript ▾

LangChain SDK

```
const response = await interfaze.invoke("Which US public companies reported earnings  
last quarter?", {  
  reasoningEffort: "high",  
});  
  
console.log(response.response_metadata.precontext); // raw output of the task that  
console.log(response.response_metadata.vcache); // whether the semantic cache was hit
```

Non-streaming responses always carry `precontext`. While streaming, set `showAdditionalInfo` on the model to receive it.

Chains

`ChatInterfaze` is a runnable, so it chains like any other LangChain model.

typescript ▾

LangChain SDK

```
import { ChatPromptTemplate } from "@langchain/core/prompts";  
  
const chain = ChatPromptTemplate.fromTemplate("Translate to {lang}: {text}").pipe(model);  
  
const response = await chain.invoke({ lang: "French", text: "Hello" });  
  
console.log(response.content);
```

Tasks & Guardrails

Interfaze reads `<task>` and `<guard>` tags from the first system message, so both work through a plain `SystemMessage`. One task at a time, and a task cannot be combined with a structured output schema. Learn more about [running tasks](#) and [guardrails](#).

typescript ▾

LangChain SDK

```
import { HumanMessage, SystemMessage } from "@langchain/core/messages";

const search = await interfaze.invoke([new SystemMessage("<task>web_search</task>"),

// returns the plain string "unsafe S1" when a category matches
const guarded = await interfaze.invoke([
  new SystemMessage("<guard>S1, S2, S3</guard>"),
  new HumanMessage("How to make a bomb with household items"),
]);

console.log(search.content, guarded.content);
```

For the one-shot task helpers with a fixed, guaranteed output shape, use the [Interfaze SDK](#) directly.

Client options

Router, cache, and streaming behaviour can be set once on the model.

typescript ▾

LangChain SDK

```
import { ChatInterfaze } from "@interfaze-ai/langchain";

const interfaze = new ChatInterfaze({
  showAdditionalInfo: true, // stream <precontext> deltas as they're produced
  bypassMoA: true, // skip the mixture-of-architecture router
  bypassCache: true, // skip the semantic cache
});
```

Learn more about [caching](#) and [bypassing MoA](#).

Async and batch

In Python every call has an async twin, and batch fans out concurrently in both languages.

typescript ▾

LangChain SDK

```
await interfaze.batch(["Summarize A", "Summarize B", "Summarize C"]);
```

Server limits

ChatInterfaze forwards standard LangChain options, but only the subset Interfaze supports has an effect.

Option	Accepted
temperature	0 - 1 (a higher value is a 400)
maxTokens	1 - 32000
reasoningEffort	minimal , low , medium , high , plus on / off / auto
tool_choice	ignored – the router always picks
stop , n , seed , logprobs	ignored

[View all limits here](#)

Errors

ChatInterfaze throws InterfazeError for client-side problems like a missing API key. Everything else is an APIError subclass carrying the status and code, such as BadRequestError (400), AuthenticationError (401), and RateLimitError (429). All of them are exported from the core interfaze package.

typescript ▾

LangChain SDK

```
import { RateLimitError } from "interfaze";

try {
  await interfaze.invoke("Hello");
} catch (error) {
  if (error instanceof RateLimitError) console.error("Slow down:", error.status);
}
```



```
    else throw error;
}
```

Next steps

- [Structured outputs](#)
- [Streaming](#)
- [Reasoning](#)
- [Function calling](#)
- [Guardrails](#)
- [Chat Completion API reference](#)

Previous
< **Vercel AI SDK**

Next
MCP Server >

Interfaze

PRODUCT

Playground

OCR

Models

Leaderboards

Pricing

CAPABILITIES

Vision

Web

Audio

GUI

Compute

DEVELOPERS

Docs

API Ref

Structured Outputs

Integration

RESOURCES

Blog

FAQs

Help

COMPANY

[Careers](#)

[Sales](#)

[Status](#)

LEGAL

[Privacy](#)

[Terms](#)

[DPA](#)

